

Tema 2: Lenguajes de script del lado del servidor

1. ¿Qué es un lenguaje de script del lado del servidor?

Un **lenguaje de script del lado del servidor** es aquel que **se ejecuta en el servidor web** antes de enviar la página al navegador del usuario.

El navegador (cliente) **solo recibe el resultado final**, normalmente en forma de **HTML, CSS y JavaScript**.

El DOM (Document Object Model)

Definición:

El **DOM (Modelo de Objetos del Documento)** es una **representación en forma de árbol** de todos los elementos de una página web (HTML o XML).

Permite que **JavaScript** u otros lenguajes accedan y modifiquen el contenido, la estructura y el estilo de la página **mientras está cargada en el navegador**.

Estructura en forma de árbol

Cada parte del documento (etiqueta, texto, atributo, etc.) se convierte en un nodo dentro de un árbol jerárquico.

Ejemplo:

```
<html>
  <body>
    <h1>Hola mundo</h1>
    <p>Este es un párrafo.</p>
  </body>
</html>
```

Representación del DOM:

```
document
├── html
│   └── body
│       ├── h1
│       │   └── "Hola mundo"
│       └── p
│           └── "Este es un párrafo."
```

Cómo se usa: gracias al DOM, JavaScript puede interactuar con la página:

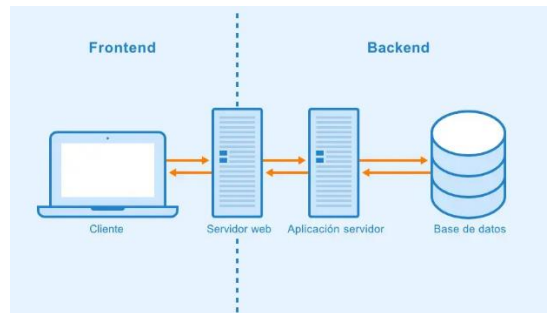
```
<script>
  document.getElementById("titulo").innerText = "Nuevo título";
</script>
```

Este script cambia el texto del elemento con **id="titulo"** sin tener que recargar la página.

En resumen:

Concepto	Explicación
DOM	Modelo de objetos del documento
Tipo	Estructura en árbol
Permite	Leer y modificar contenido, estilos y estructura
Usado por	JavaScript y otros lenguajes
Ejemplo	<code>document.querySelector("p").style.color = "blue"</code>

El frontend y el backend



Frontend (lado del cliente)

Es la parte visible de una aplicación web, lo que el usuario ve y con lo que interactúa desde el navegador.

Incluye:

- **HTML** → estructura del contenido
- **CSS** → diseño y colores
- **JavaScript** → interactividad

Se ejecuta en el navegador del usuario.

Ejemplo: botones, menús, formularios, animaciones...

Backend (lado del servidor)

Es la parte interna o lógica de la aplicación, que se ejecuta en el servidor. **No es visible para el usuario.**

Incluye:

- Lenguajes de servidor → **PHP, Python, Node.js, Java**, etc.
- Bases de datos → **MySQL, PostgreSQL, MongoDB**...
- **APIs (Application Programming Interface - Interfaz de Programación de Aplicaciones)** → **permiten comunicar el backend con el frontend.**

Se encarga de:

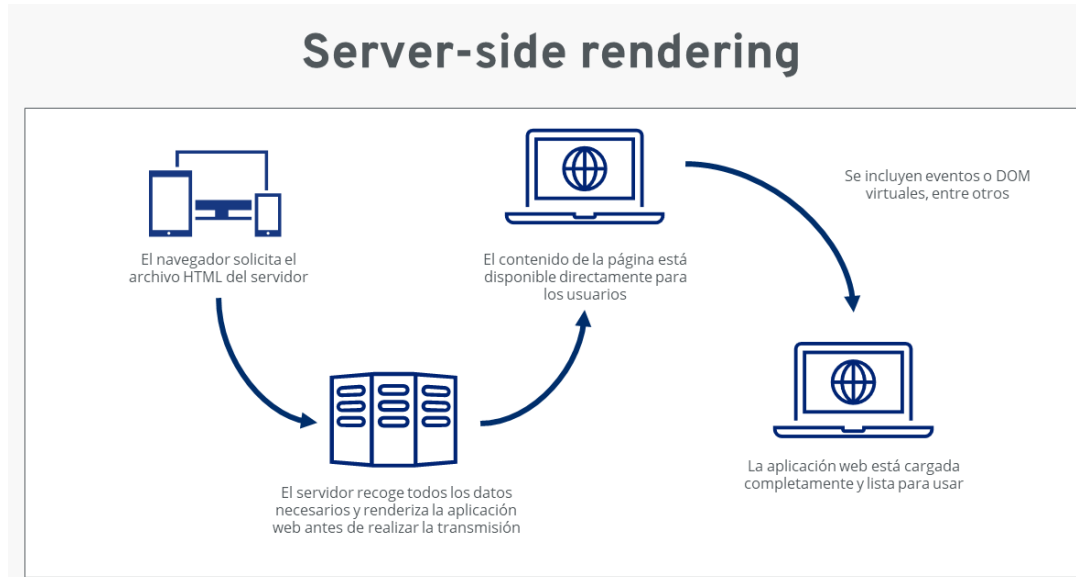
- Procesar datos de formularios
- Consultar y guardar información en la base de datos
- Controlar usuarios y seguridad

Ejemplo de flujo:

1. El usuario solicita una página: <https://miweb.com/info.php>
2. El servidor ejecuta el código **PHP** (o **Python, Node.js**, etc.)

3. El script accede si es necesario a una base de datos, procesa datos, genera contenido dinámico.
4. El servidor envía el resultado **ya procesado** al cliente.

Cliente (navegador) ← HTML generado — Servidor (con PHP, Python, etc.)



Esto **no ocurre en JavaScript del cliente**, donde el código se ejecuta dentro del navegador.

2. Características principales

Característica	Descripción
Ejecución en el servidor	El código no es visible para el usuario final.
Genera contenido dinámico	Crea páginas diferentes según el usuario, hora, datos...
Acceso a bases de datos	Puede leer, insertar o modificar datos almacenados.
Mayor seguridad	El código sensible (contraseñas, consultas SQL) no se muestra al cliente.
Integración con el servidor web	Requiere un servidor como Apache, Nginx o IIS.
Uso de plantillas o frameworks	Facilitan la creación de sitios dinámicos (Laravel, Django, Express...).

3. Tipos de lenguajes de script de servidor más utilizados:

PHP (Hypertext Preprocessor)

- Es el lenguaje de servidor más extendido en la web tradicional.
- Se integra fácilmente con **HTML** y **bases de datos MySQL**.
- Muy usado en **WordPress, Moodle, Drupal**, etc.

Ejemplo básico:

```

<!DOCTYPE html>
<html lang="es">
<head><title>Ejemplo PHP</title></head>
<body>
  <h1>Ejemplo de PHP</h1>
  <?php
    echo "Hoy es " . date("d/m/Y");
  ?>
</body>
</html>
  
```

Para poder ver el resultado deberas **instalar el módulo de PHP** para Apache (que va a ser el puente que conecta ambos). El servidor ejecuta el código PHP y **enviará al cliente solo el resultado**:

Hoy es 07/11/2025

Node.js (JavaScript del lado del servidor)

- Permite usar **JavaScript** también en el servidor.
- **Basado en el motor V8 de Google Chrome.**
- **Ideal para aplicaciones en tiempo real** (chats, juegos, APIs).
- Usa módulos y **frameworks** (es un **conjunto de librerías y plantillas** que nos ayudan a construir programas o sitios web más rápido, organizados y seguros.) como **Express.js** o **NestJS**. Mira en Internet como funcionan estos frameworks.

Ejemplo básico con Node.js y Express:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('<h1>Hola desde Node.js</h1>');
});

app.listen(3000, () => console.log('Servidor iniciado en
http://localhost:3000'));
```

Este código va a crear un servidor web básico usando **Node.js** y el **framework Express**.

Explicación línea a línea:

```
const express = require('express');
```

- Importa el **módulo Express** que es una librería de **Node.js** para crear servidores web fácilmente.
- `require('express')` carga ese módulo y lo guarda en la constante `express`.

(Equivale a importar una librería como es “**import**” en otros lenguajes).

```
const app = express();
```

- Crea una **instancia (es un objeto concreto creado a partir de una clase) de aplicación Express**, llamada `app`.
- Esta instancia representa **tu servidor web**, donde definirás rutas, peticiones, y respuestas.

```
app.get('/', (req, res) => {...});
```

- Define una **ruta HTTP GET** (sirve para **solicitar información o recursos** del servidor) para la dirección raíz (/).
- Cuando un usuario visita `http://localhost:3000/`, se ejecuta la función indicada.

Parámetros:

- **req** → (**request**-petición) contiene la solicitud del cliente (por ejemplo, navegador).
- **res** → (**response**-respuesta) permite responder al cliente.

Dentro de la función:

```
res.send('<h1>Hola desde Node.js</h1>');
```

Envía al navegador una respuesta HTML sencilla.

```
app.listen(3000, () => console.log('Servidor iniciado en http://localhost:3000'));
```

- Inicia el servidor para que escuche en el **puerto 3000**.
- La función de dentro del listen se ejecuta cuando el servidor está listo.
- Muestra un mensaje en la consola:

Servidor iniciado en http://localhost:3000

Pasos básicos para hacerlo funcionar**1. Instalar Node.js**

Desde la web oficial: <https://nodejs.org/es>

2. Verificar instalación con los comandos:

```
node -v
```

```
npm -v
```

¿Qué ocurre cuando se ejecuta?

1. Una vez que has guardado el archivo como **server.js**.
2. Abres la terminal y ejecutas:
3. **node server.js**
4. En el navegador, visitas:
5. **http://localhost:3000**
6. Verás:

Hola desde Node.js

Resumen de funcionamiento del ejemplo:

Parte	Función
<code>require('express')</code>	Importa el módulo Express
<code>express()</code>	Crea la aplicación/servidor
<code>app.get('/', ...)</code>	Define la ruta principal (inicio)
<code>res.send()</code>	Envía respuesta al navegador
<code>app.listen(3000)</code>	Inicia el servidor en el puerto 3000

Analogía con Apache + PHP:

- **Apache** = es el servidor que responde a las peticiones.
- **PHP** = es el lenguaje que genera contenido.
- En **Node.js** con **Express**, **todo lo hace el propio programa**: escucha, procesa y responde.

El lenguaje Python (con frameworks web como Django o Flask)

Python necesita **frameworks** web porque, por sí solo, no incluye las herramientas necesarias para crear aplicaciones web completas.

- **Python** es un lenguaje de propósito general muy usado en la ciencia, **IA** y web.
- Los **Frameworks** como **Django** y **Flask** permiten crear sitios completos.
- Dispone de un código legible y organizado en módulos.

Ejemplo básico con Flask:

Para trabajar con Flask debemos tener en el equipo:

1. Tener Python instalado

Flask está escrito en **Python**, así que lo primero es tener Python en tu ordenador. Puedes comprobarlo abriendo una terminal o consola y escribiendo:

```
python --version
```

o en algunos sistemas:

```
python3 --version
```

Si ves algo como **Python 3.11.5**, ya está instalado. Si no, puedes descargarlo desde <https://www.python.org/downloads>.

2. Como Instalar Flask

Flask no viene con Python por defecto, así que hay que instalarlo con el **gestor de paquetes pip**.

En el terminal escribe:

```
pip install flask
```

Esto descargará e instalará **Flask** y sus dependencias.

Puedes comprobar que se instaló correctamente con:

```
pip show flask
```

Código de ejemplo:

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def inicio():
    return "<h1>Hola desde Flask</h1>"

if __name__ == "__main__":
    app.run()
```

Cómo funciona:

```
from flask import Flask
```

- Importa la clase **Flask** del módulo **flask**.
- **Flask** es el **núcleo del framework**: permite crear aplicaciones web y gestionar las peticiones del navegador.

```
app = Flask(__name__)
```

- Crea una **instancia** de la aplicación **Flask** (tu servidor web).
- **__name__** le dice a **Flask** el **nombre del archivo actual**, útil para encontrar rutas y recursos.

Piensa en **app** como tu servidor **Flask**.

```
@app.route("/")
```

- Es un **decorador** de Python. Un decorador envuelve una función para añadirle funcionalidades extra.
- Define **una ruta web (URL)** que el servidor debe atender.
- En este caso, la ruta **/** es la **página principal** (por ejemplo, **http://localhost:5000/**).

```
def inicio():
```

- Es la **función** que se ejecuta cuando un usuario entra en la ruta **/**.
- Lo que devuelva esta función será la **respuesta que recibe el navegador**.

```
return "<h1>Hola desde Flask</h1>"
```

 Devuelve al navegador una **cadena HTML**.

- El navegador mostrará el texto como una página web.

Nota: es posible devolver etiquetas **HTML**, texto o incluso **JSON** (datos). **JSON** significa **JavaScript Object Notation** (en español, Notación de Objetos de **JavaScript**). *Es un formato de texto que se usa para guardar e intercambiar datos entre aplicaciones* (por ejemplo, entre el **frontend** y el **backend**).

En palabras simples: **JSON** sirve para enviar y recibir información estructurada, de forma sencilla y universal.

```
if __name__ == "__main__":
```

Estas instrucciones indican que el código solo se ejecuta si el archivo se ejecuta directamente (no si se importa desde otro).

```
app.run()
```

- Inicia el **servidor web local** de **Flask**.
- Por defecto usa el **puerto 5000** y la dirección **localhost**.

Resultado:

Al abrir en el navegador y poner la URL: **http://127.0.0.1:5000/** veremos:

Hola desde Flask

Resumen:

Parte del código	Función
<code>Flask(__name__)</code>	Crea la aplicación web
<code>@app.route("/")</code>	Define la URL de inicio
<code>def inicio()</code>	Función que devuelve la respuesta
<code>app.run()</code>	Arranca el servidor local

Una analogía sencilla:

- **Flask** = sería el restaurante.
- **@app.route("/")** = es el camarero que atiende una mesa específica (una **URL** /).
- **def inicio()** = es la cocina que prepara la comida (respuesta).
- **return "<h1>Hola...</h1>"** = el plato que se sirve al cliente (**HTML**).

El lenguaje ASP.NET (Microsoft)

- Desarrollado por Microsoft con lo que toda la infraestructura debe de ser de tipo Windows NT.
- Utiliza **C#** o **VB.NET**.
- Se ejecuta sobre servidores **IIS** (Internet Information Services).
- Muy usado en entornos empresariales.

Ejemplo:

```
@{
    var fecha = DateTime.Now;
}
<h1>Fecha actual: @fecha</h1>
```

Este fragmento de código es una **vista Razor**, el motor de plantillas que usa **ASP.NET** para mezclar **código C#** con **HTML**.

- El símbolo **@** indica al servidor que lo que sigue es **código C#**, no texto **HTML**.
- El servidor **ejecuta ese código en el servidor**, genera el resultado (por ejemplo, la fecha actual), y **envía HTML puro al navegador**.

Línea por línea

```
@{
    var fecha = DateTime.Now;
}
```

Las llaves **{ }** contienen código C#.

Se declara una **variable** llamada fecha.

DateTime.Now obtiene la fecha y hora actual del sistema del servidor. Por ejemplo: **06/11/2025 18:42:11**.

Este código no se ve en el navegador, solo se ejecuta en el servidor.


```
<h1>Fecha actual: @fecha</h1>
```

Aquí hay código HTML esta mezclado con una variable de C#.

El símbolo @ antes de fecha le dice al motor Razor: "inserta aquí el valor de la variable fecha".

El resultado que el navegador recibe será:

```
<h1>Fecha actual: 07/11/2025 18:42:11</h1>
```

El navegador solo ve el HTML generado, no el código C#.

Cómo funciona internamente:

1. El usuario entra en la página (por ejemplo, <http://localhost:5000/fecha>).
2. El servidor web (IIS o Kestrel) detecta que hay código Razor (@...).
3. ASP.NET ejecuta el código C# dentro de las llaves.
4. El servidor genera una página HTML con el resultado.
5. El navegador recibe solo el HTML final.

En resumen:

Elemento	Función
@{ ... }	Bloque de código C# que se ejecuta en el servidor
@variable	Inserta el valor de la variable en el HTML
DateTime.Now	Devuelve la fecha y hora actuales
Resultado:	HTML con el texto ya generado

La salida final en el navegador:

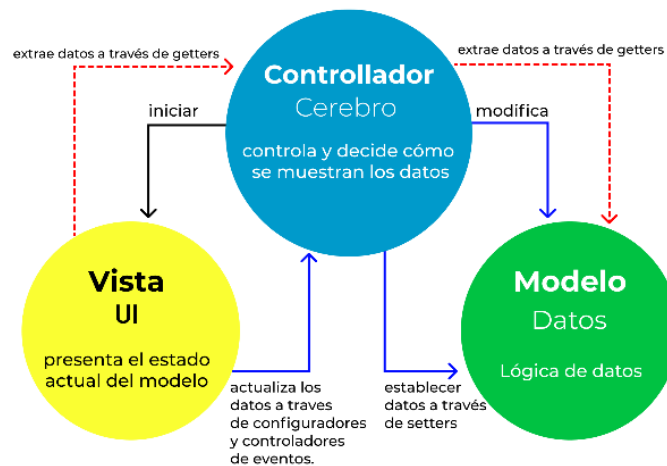
```
<h1>Fecha actual: 07/11/2025 18:42:11</h1>
```

Lenguaje Ruby (con Ruby on Rails)

Ruby on Rails (abreviado *Rails*) es un **framework web MVC** (*Modelo–Vista–Controlador*) que permite crear aplicaciones web de forma rápida y estructurada con el lenguaje **Ruby**.

- Es un lenguaje elegante y conciso.
- **Ruby on Rails facilita el desarrollo rápido de aplicaciones web.**
- Usa el patrón **MVC** (Modelo-Vista-Controlador):

Patrones de Arquitectura MVC



Ejemplo:

Este código define un **controlador**, que es la parte del programa que recibe las peticiones del navegador y decide qué respuesta enviar.

```
class BienvenidaController < ApplicationController
  def index
    render html: "<h1>Hola desde Ruby on Rails</h1>".html_safe
  end
end
```

Explicación línea por línea

```
class BienvenidaController < ApplicationController
```

- Declara una **clase** llamada **BienvenidaController**.
- El nombre indica que es un **controlador** (**Controller**) encargado de gestionar las rutas relacionadas con "Bienvenida".
- El símbolo < significa **herencia**:
Es decir que **BienvenidaController** hereda de **ApplicationController**, que contiene las **funciones básicas** de todos los **controladores** en **Rails**.

Nota: en **Rails**, todos los controladores heredan de **ApplicationController** para tener acceso a funciones como **render**, **redirect_to**, **params**, etc.

```
def index
```

- Define un **método** (función) llamado **index**.
- En **Rails**, el método **index** suele representar la **página principal** de un recurso o controlador.
- Este método se ejecuta cuando el usuario visita la ruta **/bienvenida** o **/bienvenida/index**.

```
render html: "<h1>Hola desde Ruby on Rails</h1>".html_safe
```

Donde **render** es un **método** que **envía una respuesta al navegador**.
Es decir le estamos indicando a **Rails**:

"Muestra esta cadena HTML como respuesta".

- **html:** **"<h1>Hola desde Ruby on Rails</h1>"** indica el contenido que se mostrará.
- **.html_safe** le dice a **Rails** que **no escape** el HTML ya que, por defecto, Rails convierte los caracteres especiales en texto plano por seguridad, pero **.html_safe** permite que el navegador interprete las etiquetas **<h1>** correctamente.

Sin **.html_safe**: se vería como texto literal: **<h1>Hola desde Ruby on Rails</h1>**

Sin embargo al usar: **.html_safe**: se renderiza como: **Hola desde Ruby on Rails**

```
end
```

- Cierra el método **index**.

El ultimo: **end**

- Cierra la clase **BienvenidaController**.

Cómo funciona al ejecutarse

1. El usuario entra a <http://localhost:3000/bienvenida>
2. **Rails** detecta la ruta y llama al método **index** del controlador **BienvenidaController**.
3. Ese método ejecuta **render html:** ...
4. El servidor envía el HTML generado al navegador.
5. El navegador muestra:

Hola desde Ruby on Rails

Resumen:

Elemento	Función
class BienvenidaController	Define un controlador web en Rails
< ApplicationController	Hereda funciones básicas del framework
def index	Método que maneja la ruta principal
render html:	Envía una respuesta HTML al navegador
.html_safe	Permite mostrar etiquetas HTML reales
Resultado	<h1>Hola desde Ruby on Rails</h1>

En resumen:

Este código crea un **controlador en Ruby on Rails** que, al acceder a la página **/bienvenida**, **muestra un título HTML generado desde el servidor**.

4. Comparativa general de los lenguajes:

Lenguaje	Velocidad	Facilidad de uso	Comunidad	Bases de datos comunes	Frameworks populares
PHP	Media-Alta	Alta	Muy amplia	MySQL, MariaDB	Laravel, Symfony
Node.js	Muy alta	Media	Muy activa	MongoDB, PostgreSQL	Express, NestJS
Python	Media	Alta	Muy amplia	SQLite, PostgreSQL, MySQL	Django, Flask
ASP.NET	Alta	Media	Amplia (entorno Windows)	SQL Server	ASP.NET Core
Ruby	Media	Media	Menor que PHP/JS	PostgreSQL, MySQL	Ruby on Rails

5. Ventajas y desventajas generales

Ventajas

- Permiten generar contenido dinámico.
- Acceso directo a bases de datos.
- Código oculto al cliente.
- Pueden combinarse con HTML, CSS y JavaScript.

Desventajas

- Requieren servidor con intérprete o motor adecuado.
- Consumen recursos del servidor.
- Mayor complejidad que las páginas estáticas.

6. Relación con la arquitectura cliente-servidor

Cliente (navegador)	Servidor (back-end)
HTML, CSS, JS (visual)	PHP, Python, Node.js, etc. (lógica)
Envía peticiones HTTP	Procesa datos y genera respuestas
Ejecuta código en el navegador	Ejecuta código en el servidor

Intercambio a través del protocolo HTTP:

- El cliente → Solicita /productos
- El servidor → Genera el HTML con datos de los productos y responde

7. Conclusión

Los **lenguajes de script del lado del servidor** son esenciales para crear **aplicaciones web dinámicas, seguras e interactivas**.

Aunque sólo vemos HTML, CSS y JavaScript en el lado cliente cuando comenzamos a aprender la asignatura, el conocimiento de estos otros lenguajes nos va a permitir dar el salto hacia el **desarrollo completo (Full Stack)**.